

Dynamic Programming Analysis: The 0-1 Knapsack Problem

Algorithm Implementation and Analysis

June 28, 2025

Contents

1	Problem Statement	3
1.1	Definition	3
1.2	Real-World Analogy	3
1.3	Mathematical Formulation	3
2	Problem Visualization	3
2.1	Example Setup	3
2.2	Decision Tree Visualization	4
3	Solution Approaches	5
3.1	Approach 1: Naive Recursion	5
3.1.1	Algorithm	5
3.1.2	Java Implementation	5
3.1.3	Complexity Analysis	6
3.2	Approach 2: Memoization (Top-Down DP)	6
3.2.1	Key Insight	6
3.2.2	Memoization Table	6
3.2.3	Algorithm	6
3.2.4	Java Implementation	6
3.2.5	Complexity Analysis	8
3.3	Approach 3: Tabulation (Bottom-Up DP)	8
3.3.1	Key Insight	8
3.3.2	Table Interpretation	8
3.3.3	Recurrence Relation	8
3.3.4	Step-by-Step Table Construction	8
3.3.5	Algorithm	9
3.3.6	Java Implementation	9
3.3.7	Complexity Analysis	10
4	Detailed Example Walkthrough	10
4.1	Filling <code>dp[1][w]</code> (First item: Tent)	10
4.2	Filling <code>dp[2][30]</code> (Adding sleeping bag, capacity 30)	10
4.3	Filling <code>dp[3][50]</code> (Adding food, capacity 50)	10

5	Finding the Actual Items Selected	11
6	Space Optimization	11
7	Comparison of Approaches	11
8	Variations and Extensions	11
8.1	Fractional Knapsack	11
8.2	Unbounded Knapsack	12
8.3	Multiple Knapsacks	12
8.4	Bounded Knapsack	12
9	Real-World Applications	12
10	Common Pitfalls and Debugging Tips	12
10.1	Array Indexing	12
10.2	Base Cases	12
10.3	Integer Overflow	13
10.4	Testing Strategy	13
11	Advanced Topics	13
11.1	Branch and Bound	13
11.2	Approximation Algorithms	13
11.3	Parallel Implementation	13
12	Conclusion	13
13	References	14

1 Problem Statement

1.1 Definition

The **0-1 Knapsack Problem** is a classic optimization problem that asks:

Given a knapsack with a fixed weight capacity and a collection of items, each with a specific weight and value, determine the most valuable combination of items that can fit in the knapsack. Each item can be either included (1) or excluded (0), hence the name "0-1" knapsack.

1.2 Real-World Analogy

Imagine you're going on a hiking trip with a backpack that can carry only 50 pounds. You have various items to choose from:

- Tent: 10 lbs, Value: \$60 (essential shelter)
- Sleeping bag: 20 lbs, Value: \$100 (comfort and warmth)
- Food supplies: 30 lbs, Value: \$120 (nutrition for the trip)

You need to maximize the total value while staying within the weight limit. This is exactly what the 0-1 Knapsack problem solves!

1.3 Mathematical Formulation

Let:

- n = number of items
- W = knapsack capacity
- w_i = weight of item i
- v_i = value of item i
- $x_i \in \{0, 1\}$ = decision variable (0 = exclude, 1 = include)

Objective: Maximize $\sum_{i=1}^n v_i \cdot x_i$

Subject to: $\sum_{i=1}^n w_i \cdot x_i \leq W$

2 Problem Visualization

2.1 Example Setup

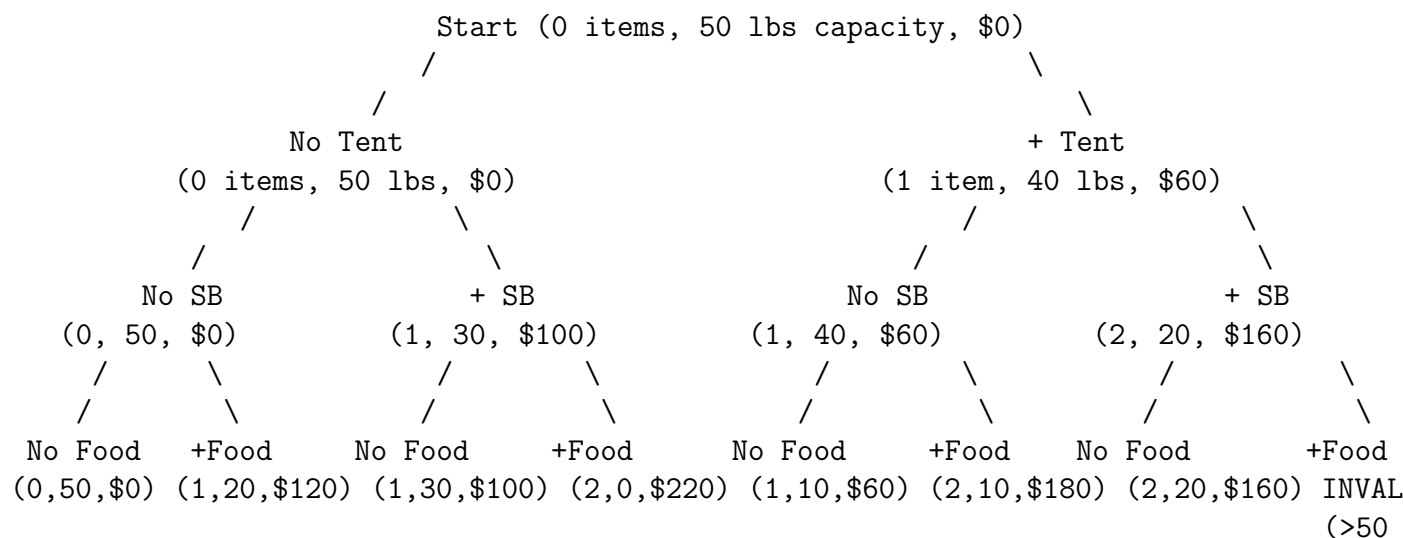
Let's use our hiking example:

Item	Weight (lbs)	Value (\$)	Value/Weight
Tent	10	60	6.0
Sleeping Bag	20	100	5.0
Food Supplies	30	120	4.0

Capacity: 50 lbs

2.2 Decision Tree Visualization

The decision tree for our 0-1 Knapsack problem can be represented as follows:



Final Solutions Summary:

Combination	Total Weight	Total Value
Empty knapsack	0 lbs	\$0
Food only	30 lbs	\$120
Sleeping bag only	20 lbs	\$100
Sleeping bag + Food	50 lbs	\$220 <- OPT
Tent only	10 lbs	\$60
Tent + Food	40 lbs	\$180
Tent + Sleeping bag	30 lbs	\$160
All items (T+SB+F)	60 lbs	INVALID

Figure 1: Decision tree showing all possible combinations for the 0-1 Knapsack problem. Each node shows (items count, remaining capacity, total value). The optimal solution is Sleeping Bag + Food = \$220.

Key Observations:

- Each level represents a decision about one item (Tent, Sleeping Bag, Food)
- Left branches represent "exclude item", right branches represent "include item"
- The optimal solution uses exactly the full capacity (50 lbs)
- Multiple valid solutions exist, but \$220 is the maximum value achievable

3 Solution Approaches

3.1 Approach 1: Naive Recursion

3.1.1 Algorithm

Algorithm 1 0-1 Knapsack - Recursive Approach

Require: Arrays *weights*[], *values*[], capacity *W*, number of items *n*

Ensure: Maximum value that can be obtained

```

function knapsackRecursion(weights, values, W, n)
  if n = 0 OR W = 0 then
    return 0
  end if
  if weights[n - 1] ≤ W then
    include = values[n - 1] + knapsackRecursion(weights, values, W - weights[n - 1],
    n - 1)
    exclude = knapsackRecursion(weights, values, W, n - 1)
    return max(include, exclude)
  else
    return knapsackRecursion(weights, values, W, n - 1)
  end if

```

3.1.2 Java Implementation

Listing 1: Naive Recursive Solution

```

1 static int knapsackRecursion(int[] weights, int[] values, int
  capacity, int n) {
2     // Base case: no items left or capacity is 0
3     if (n == 0 || capacity == 0) {
4         return 0;
5     }
6
7     // If the weight of the nth item is less than the capacity
8     if (weights[n - 1] <= capacity) {
9         // Include the nth item and check the value with and
10        without it
11        int includeItem = values[n - 1] + knapsackRecursion(
12            weights, values,
13            capacity - weights[n - 1], n - 1);
14        int excludeItem = knapsackRecursion(weights, values,
15            capacity, n - 1);
16        return Math.max(includeItem, excludeItem);
17    } else {
18        // If the nth item cannot be included, skip it
19        return knapsackRecursion(weights, values, capacity, n -
20            1);
21    }
22 }

```

3.1.3 Complexity Analysis

Time Complexity: $O(2^n)$ - Each item has 2 choices (include/exclude)

Space Complexity: $O(n)$ - Recursion stack depth

Problems:

- Exponential time complexity makes it impractical for large inputs
- Redundant calculations of same subproblems
- Stack overflow for large n

3.2 Approach 2: Memoization (Top-Down DP)

3.2.1 Key Insight

The recursive solution recalculates the same subproblems multiple times. For example, the state $(n = 2, \text{capacity} = 30)$ might be computed several times in different branches.

3.2.2 Memoization Table

We use a 2D table $memo[n][capacity]$ where:

- $memo[i][w] = \text{maximum value using first } i \text{ items with capacity } w$
- Initially filled with *null* (or -1) to indicate uncomputed states

Items	0	null	null	null	null	null
	1	null	60	60	60	60
	2	null	60	100	160	160
	3	null	60	100	160	180
		0	10	20	30	40
						50
						Capacity

Figure 2: Memoization table being filled during computation

3.2.3 Algorithm

3.2.4 Java Implementation

Listing 2: Memoization Solution

```

1 static int KnapsackMemoization(int[] weights, int[] values, int
   capacity,
2                                     int n, Integer[][] d) {
3     // Base case: no items left or capacity is 0

```

Algorithm 2 0-1 Knapsack - Memoization

Require: Arrays *weights*[], *values*[], capacity *W*, number of items *n*, memo table *memo*[][]

Ensure: Maximum value that can be obtained

```

function knapsackMemo(weights, values, W, n, memo)
  if n = 0 OR W = 0 then
    return 0
  end if
  if memo[n][W] ≠ null then
    return memo[n][W]
  end if
  if weights[n - 1] ≤ W then
    include = values[n - 1] + knapsackMemo(weights, values, W - weights[n - 1], n - 1, memo)
    exclude = knapsackMemo(weights, values, W, n - 1, memo)
    memo[n][W] = max(include, exclude)
  else
    memo[n][W] = knapsackMemo(weights, values, W, n - 1, memo)
  end if
  return memo[n][W]

```

```

4      if (n == 0 || capacity == 0) {
5          return 0;
6      }
7
8      // If the value is already computed, return it
9      if (d[n][capacity] != null) {
10         return d[n][capacity];
11     }
12
13     // If the weight of the nth item is less than or equal to the
14     // capacity
15     if (weights[n - 1] <= capacity) {
16         // Include the nth item and check the value with and
17         // without it
18         int includeItem = values[n - 1] + KnapsackMemoization(
19             weights, values,
20             capacity - weights[n - 1], n - 1, d);
21         int excludeItem = KnapsackMemoization(weights, values,
22             capacity, n - 1, d);
23         d[n][capacity] = Math.max(includeItem, excludeItem);
24     } else {
25         // If the nth item cannot be included, skip it
26         d[n][capacity] = KnapsackMemoization(weights, values,
27             capacity, n - 1, d);
28     }
29     return d[n][capacity];
30 }

```

3.2.5 Complexity Analysis

Time Complexity: $O(n \times W)$ - Each subproblem solved exactly once

Space Complexity: $O(n \times W)$ - Memoization table + $O(n)$ recursion stack

3.3 Approach 3: Tabulation (Bottom-Up DP)

3.3.1 Key Insight

Instead of recursion, we build the solution iteratively from the ground up. We systematically fill a 2D table where each cell represents the optimal solution for a subproblem.

3.3.2 Table Interpretation

$dp[i][w]$ = "Maximum value achievable using the first i items with knapsack capacity w "

3.3.3 Recurrence Relation

$$dp[i][w] = \begin{cases} 0 & \text{if } i = 0 \text{ or } w = 0 \\ dp[i-1][w] & \text{if } weights[i-1] > w \\ \max(dp[i-1][w], dp[i-1][w - weights[i-1]] + values[i-1]) & \text{otherwise} \end{cases} \quad (1)$$

3.3.4 Step-by-Step Table Construction

Step 1: Initialize base cases

$i \backslash w$	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	?	?	?	?	?
2	0	?	?	?	?	?
3	0	?	?	?	?	?

Step 2: Fill row 1 (Item 1: weight=10, value=60)

$i \backslash w$	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	?	?	?	?	?
3	0	?	?	?	?	?

Step 3: Fill row 2 (Item 2: weight=20, value=100)

$i \backslash w$	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	?	?	?	?	?

Step 4: Fill row 3 (Item 3: weight=30, value=120)

i\w	0	10	20	30	40	50
0	0	0	0	0	0	0
1	0	60	60	60	60	60
2	0	60	100	160	160	160
3	0	60	100	160	180	220

3.3.5 Algorithm

Algorithm 3 0-1 Knapsack - Tabulation

Require: Arrays *weights*[], *values*[], capacity *W*

Ensure: Maximum value that can be obtained

$n \leftarrow$ length of *weights*[]

Create 2D array *dp*[*n* + 1][*W* + 1] initialized to 0

for *i* = 1 to *n* **do**

for *w* = 0 to *W* **do**

if *weights*[*i* - 1] ≤ *w* **then**

dp[*i*][*w*] ← max(*dp*[*i* - 1][*w*], *dp*[*i* - 1][*w* - *weights*[*i* - 1]] + *values*[*i* - 1])

else

dp[*i*][*w*] ← *dp*[*i* - 1][*w*]

end if

end for

end for

return *dp*[*n*][*W*]

3.3.6 Java Implementation

Listing 3: Tabulation Solution

```

1 static int KnapsackTabulation(int[] weights, int[] values, int
  capacity) {
2     int n = weights.length;
3     int[][] dp = new int[n + 1][capacity + 1];
4
5     // dp[i][w] will store the maximum value that can be obtained
6     // with the first i items and capacity w
7
8     // Build the dp table in bottom-up manner
9     for (int i = 1; i <= n; i++) {
10        for (int w = 0; w <= capacity; w++) {
11            if (weights[i - 1] <= w) {
12                dp[i][w] = Math.max(dp[i - 1][w],
13                                    dp[i - 1][w - weights[i - 1]] + values[
14                                        i - 1]);
15            } else {
16                dp[i][w] = dp[i - 1][w];
17            }
18        }
19    }
20 }

```

```

19     return dp[n][capacity];
20 }

```

3.3.7 Complexity Analysis

Time Complexity: $O(n \times W)$ - Two nested loops

Space Complexity: $O(n \times W)$ - 2D DP table

4 Detailed Example Walkthrough

Let's trace through our hiking example step by step using tabulation:

Given:

- Items: Tent (10 lbs, \$60), Sleeping Bag (20 lbs, \$100), Food (30 lbs, \$120)
- Capacity: 50 lbs

4.1 Filling $dp[1][w]$ (First item: Tent)

For each capacity w from 0 to 50:

- $w < 10$: Cannot include tent, so $dp[1][w] = dp[0][w] = 0$
- $w \geq 10$: Can include tent
 - Include: $dp[0][w - 10] + 60 = 0 + 60 = 60$
 - Exclude: $dp[0][w] = 0$
 - Choose max: $dp[1][w] = 60$

4.2 Filling $dp[2][30]$ (Adding sleeping bag, capacity 30)

- Can we include sleeping bag (weight 20)? Yes, $20 \leq 30$
- Include sleeping bag: $dp[1][30 - 20] + 100 = dp[1][10] + 100 = 60 + 100 = 160$
- Exclude sleeping bag: $dp[1][30] = 60$
- Choose max: $dp[2][30] = \max(160, 60) = 160$

4.3 Filling $dp[3][50]$ (Adding food, capacity 50)

- Can we include food (weight 30)? Yes, $30 \leq 50$
- Include food: $dp[2][50 - 30] + 120 = dp[2][20] + 120 = 100 + 120 = 220$
- Exclude food: $dp[2][50] = 160$
- Choose max: $dp[3][50] = \max(220, 160) = 220$

Note: The value 220 represents *Sleeping Bag + Food* ($20 + 30 = 50$ lbs, exactly at capacity). This is the best combination. *Food* ($10 + 30 = 40$ lbs, 180) is another valid combination that uses less capacity.

Final Answer: The maximum value is \$220 (Sleeping Bag + Food combination).

5 Finding the Actual Items Selected

To find which items were selected, we trace back through the DP table:

Algorithm 4 Backtracking to Find Selected Items

```

 $i \leftarrow n, w \leftarrow W$ 
Create empty list selectedItems
while  $i > 0$  AND  $w > 0$  do
  if  $dp[i][w] \neq dp[i-1][w]$  then
    Add item  $i$  to selectedItems
     $w \leftarrow w - weights[i-1]$ 
  end if
   $i \leftarrow i - 1$ 
end while
return selectedItems

```

6 Space Optimization

Since each row only depends on the previous row, we can optimize space to $O(W)$:

Listing 4: Space-Optimized Tabulation

```

1 static int KnapsackOptimized(int[] weights, int[] values, int
  capacity) {
2   int n = weights.length;
3   int[] dp = new int[capacity + 1];
4
5   for (int i = 0; i < n; i++) {
6     // Traverse from right to left to avoid using updated
       values
7     for (int w = capacity; w >= weights[i]; w--) {
8       dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]
          );
9     }
10  }
11  return dp[capacity];
12 }

```

Key Insight: We traverse from right to left to ensure we don't use already updated values from the current iteration.

7 Comparison of Approaches

8 Variations and Extensions

8.1 Fractional Knapsack

Unlike 0-1 Knapsack, items can be broken into fractions. This can be solved greedily by value-to-weight ratio.

Approach	Time	Space	Practical Limit	Pros/Cons
Naive Recursion	$O(2^n)$	$O(n)$	$n \leq 20$	Simple but inefficient
Memoization	$O(n \times W)$	$O(n \times W)$	$n, W \leq 1000$	Top-down, intuitive
Tabulation	$O(n \times W)$	$O(n \times W)$	$n, W \leq 1000$	Bottom-up, iterative
Space Optimized	$O(n \times W)$	$O(W)$	$W \leq 10^6$	Memory efficient

Table 1: Comparison of different approaches

8.2 Unbounded Knapsack

Each item can be used multiple times. The recurrence becomes:

$$dp[i][w] = \max(dp[i-1][w], dp[i][w - weights[i-1]] + values[i-1])$$

8.3 Multiple Knapsacks

Multiple knapsacks with different capacities - requires 3D DP.

8.4 Bounded Knapsack

Each item has a limited quantity available.

9 Real-World Applications

1. **Resource Allocation:** Budget allocation across projects
2. **Portfolio Selection:** Investment selection with budget constraints
3. **Cargo Loading:** Optimizing cargo in ships/planes
4. **Memory Management:** Selecting data structures for limited memory
5. **Feature Selection:** Machine learning model optimization
6. **Network Optimization:** Bandwidth allocation

10 Common Pitfalls and Debugging Tips

10.1 Array Indexing

- Remember: $weights[i-1]$ and $values[i-1]$ when i represents the item number
- DP table indices: $dp[i][w]$ where i goes from 0 to n and w from 0 to W

10.2 Base Cases

- $dp[0][w] = 0$ for all w (no items = no value)
- $dp[i][0] = 0$ for all i (no capacity = no value)

10.3 Integer Overflow

For large values, use `long` instead of `int`.

10.4 Testing Strategy

1. Test with simple cases (single item, zero capacity)
2. Verify against brute force for small inputs
3. Check edge cases (all items too heavy, all items fit)

11 Advanced Topics

11.1 Branch and Bound

For exact solutions with better average-case performance than DP.

11.2 Approximation Algorithms

When exact solutions are too expensive:

- Greedy by value-to-weight ratio
- FPTAS (Fully Polynomial-Time Approximation Scheme)

11.3 Parallel Implementation

DP table can be filled diagonally for parallel processing.

12 Conclusion

The 0-1 Knapsack problem is a cornerstone of dynamic programming, demonstrating:

- **Optimal Substructure:** Optimal solution contains optimal solutions to subproblems
- **Overlapping Subproblems:** Same subproblems appear in different branches
- **Trade-offs:** Time vs. space complexity considerations
- **Problem Transformation:** Converting recursive solutions to iterative DP

Key Takeaways:

- Always start with recursive solution to understand the problem structure
- Identify overlapping subproblems for memoization opportunities
- Consider space optimization when memory is a constraint
- Understand the problem constraints to choose the right approach

Learning Objectives Achieved:

- Understanding classical optimization problems
- Implementing three DP paradigms (recursion, memoization, tabulation)
- Analyzing time and space complexity trade-offs
- Recognizing real-world applications of algorithmic concepts
- Developing debugging and testing strategies for DP problems

13 References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Kleinberg, J., & Tardos, E. (2005). *Algorithm Design*. Addison-Wesley.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.
4. Dasgupta, S., Papadimitriou, C., & Vazirani, U. (2006). *Algorithms*. McGraw-Hill.
5. Dynamic Programming Problems and Applications in Computer Science